

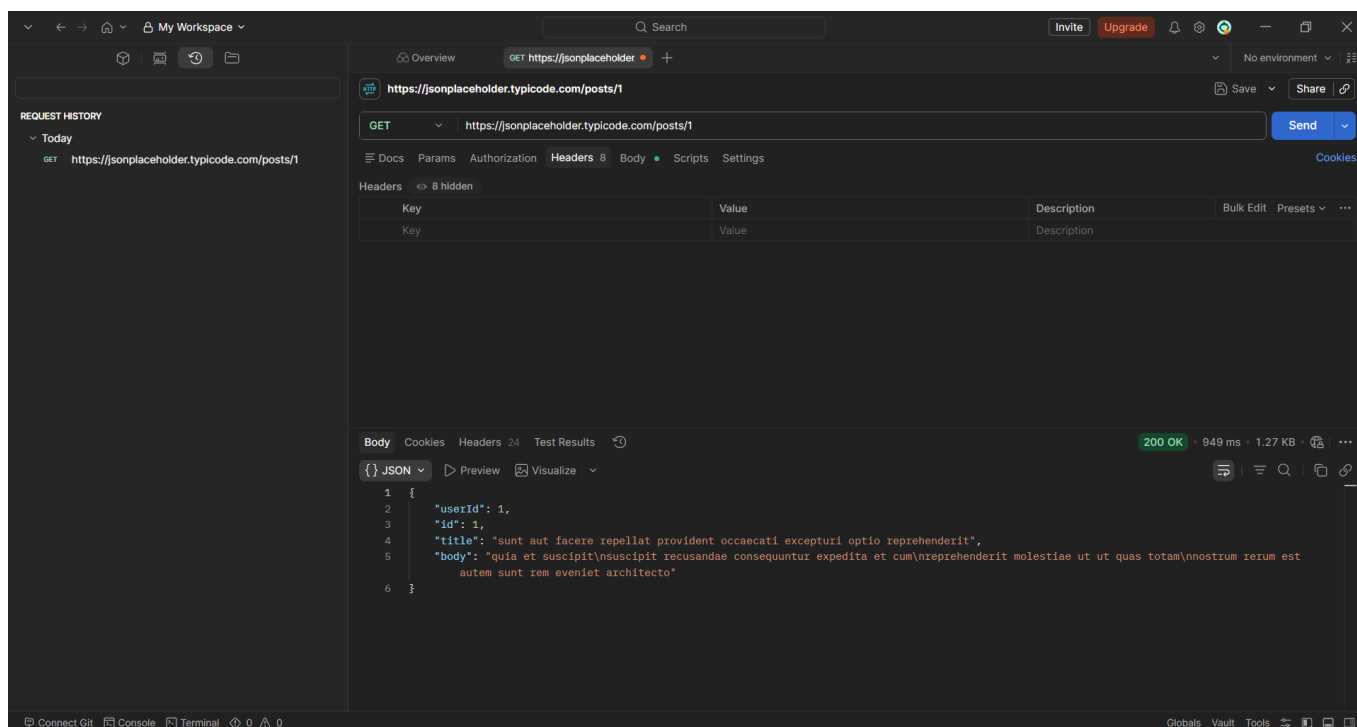
Practice Assignment

API Fundamentals & Postman Mastery

TASKS

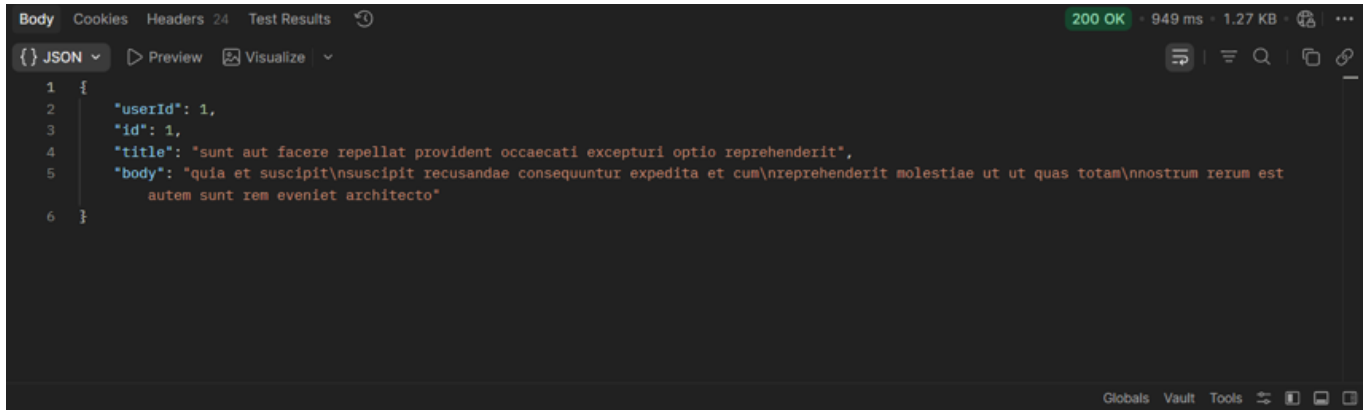
TASK 1 Reading Data Without Headers

a) Send a GET request to `/posts/1` — leave the Headers and Body tabs completely empty.



b) Record the status code you get back.

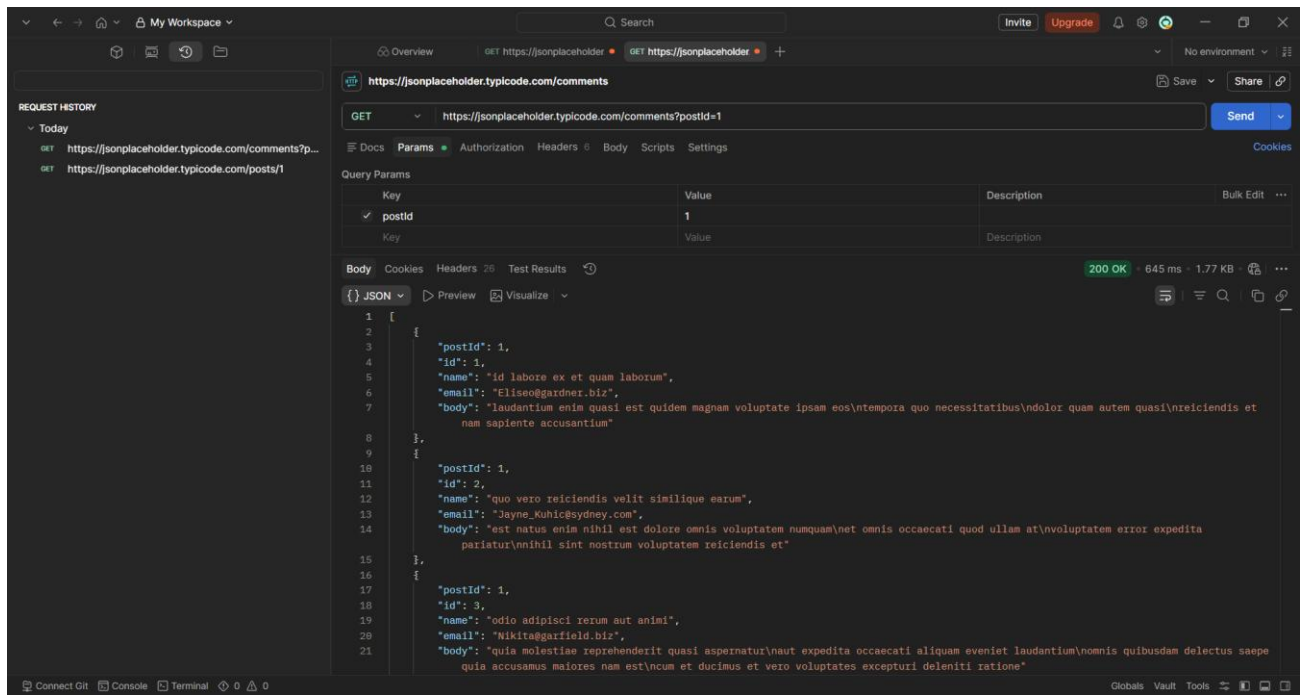
NEEL VAGHELA



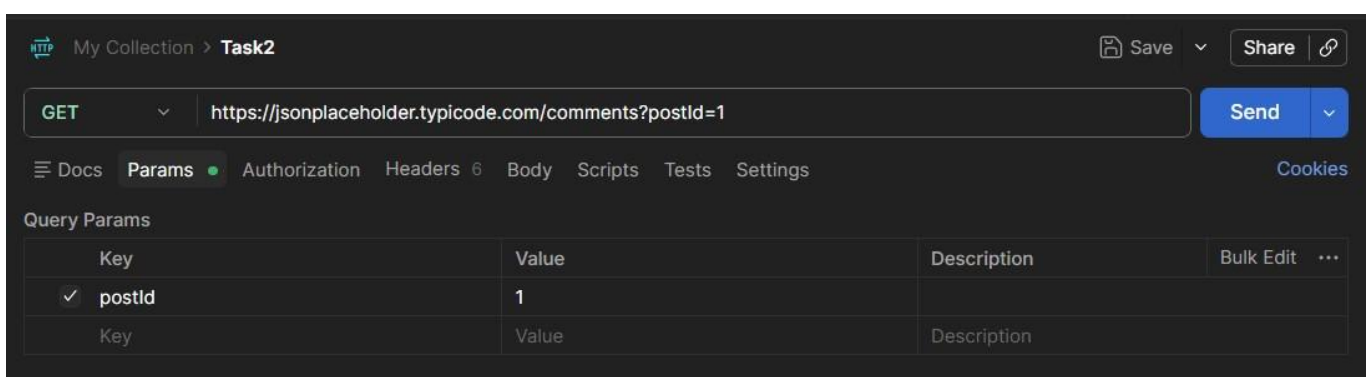
- c) In one sentence, explain why this request works even with nothing in the Headers tab.
- Because GET method is used to retrieve data and it is only method without header so we not need to pass any header it work efficiently without any problem.

TASK 2 Filtering With Query Parameters

a) Send a GET request to /comments, and add query parameter in Params tab: postId = 1.



b) Confirm the URL automatically updates to include ?postId=1.



c) Now combine two filters at once on /todos: userId = 1 and completed = false.

The screenshot shows a REST client interface with the following details:

- URL:** `https://jsonplaceholder.typicode.com/todos`
- Method:** GET
- Query Params:**

Key	Value	Description
userId	1	
completed	false	
- Status:** 200 OK
- Response Body (JSON):**

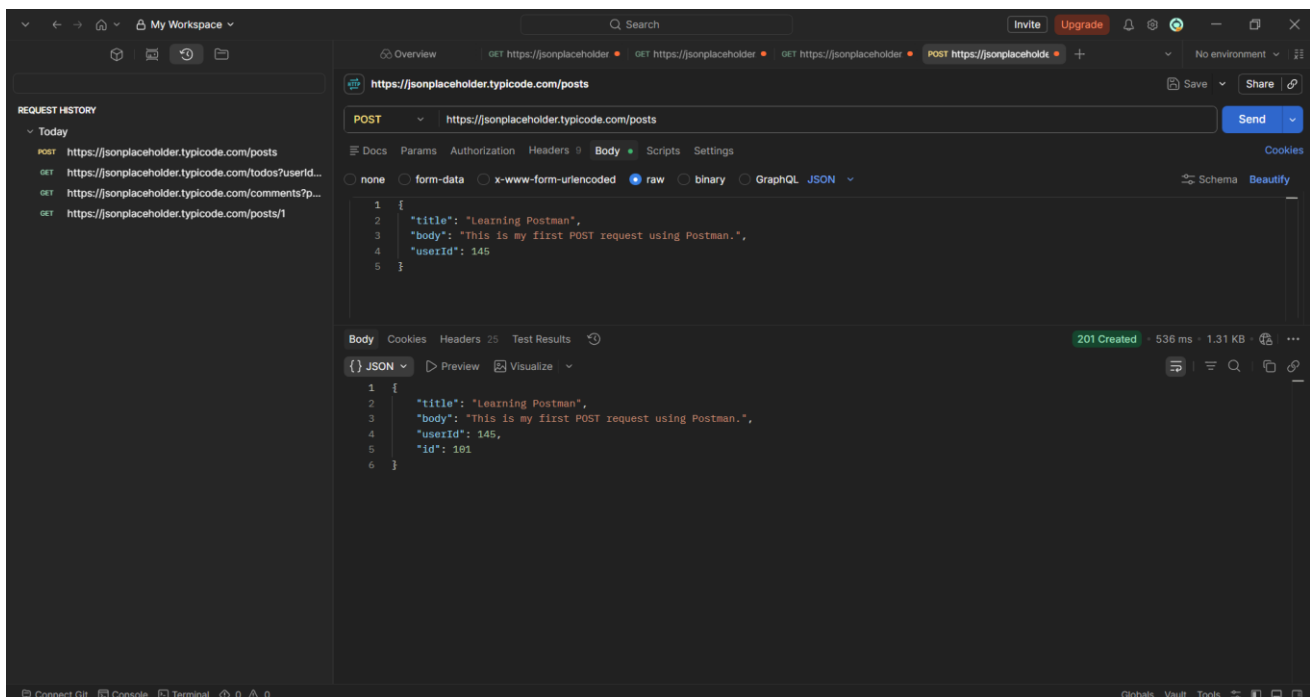
```
[
  {
    "userId": 1,
    "id": 1,
    "title": "delectus aut autem",
    "completed": false
  },
  {
    "userId": 1,
    "id": 2,
    "title": "quis ut nam facilis et officia qui",
    "completed": false
  },
  {
    "userId": 1,
    "id": 3,
    "title": "fugiat veniam minus",
    "completed": false
  }
]
```

- d) In one sentence, explain what changed in the results compared to not using any params.
- Using query parameters filters the data returned by the server, so only records matching the specified conditions are included instead of all records

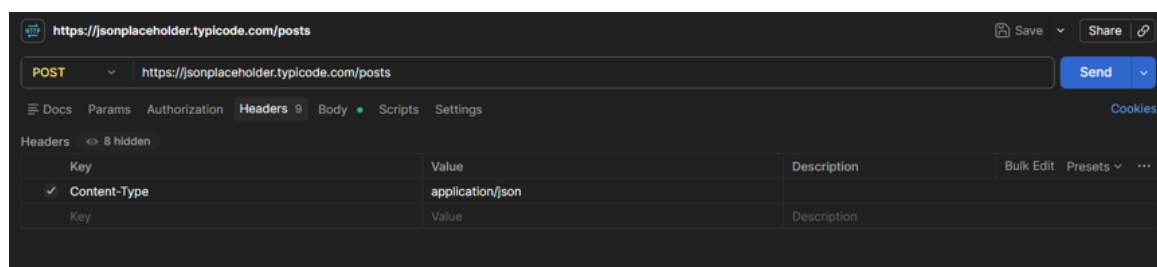
NEEL VAGHELA

TASK 3 Sending Data With Headers & a Body

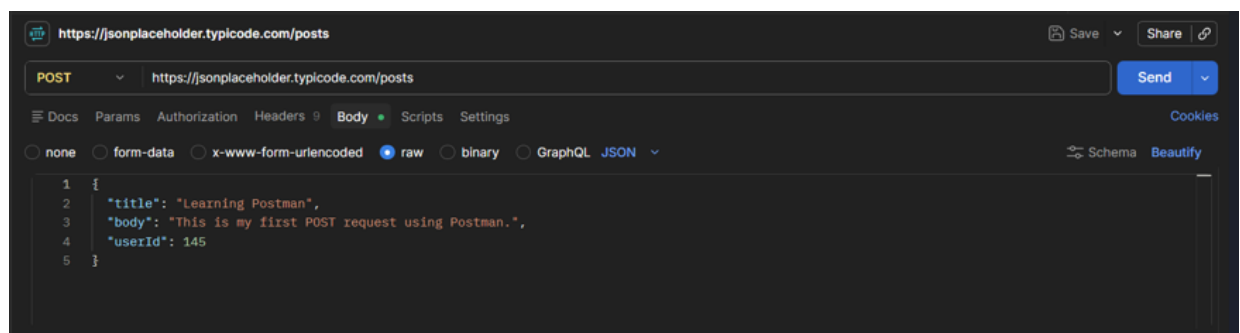
a) Send a POST request to /posts.



b) In the Headers tab, add: Content-Type: application/json

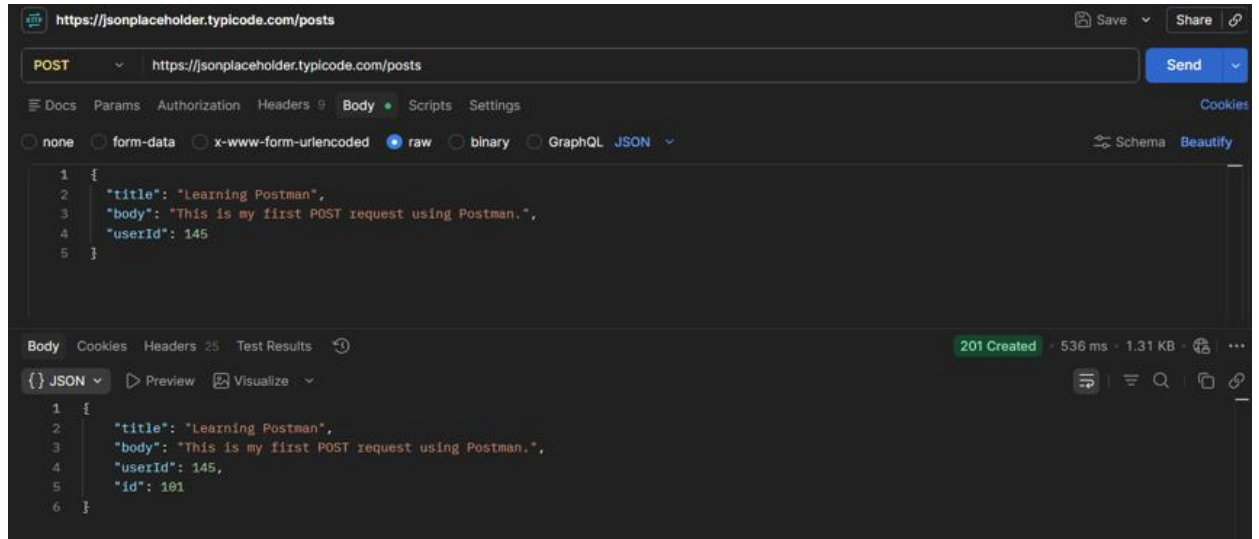


c) In the Body tab (raw, JSON), send a new post with a title, body, and userId of your choice.



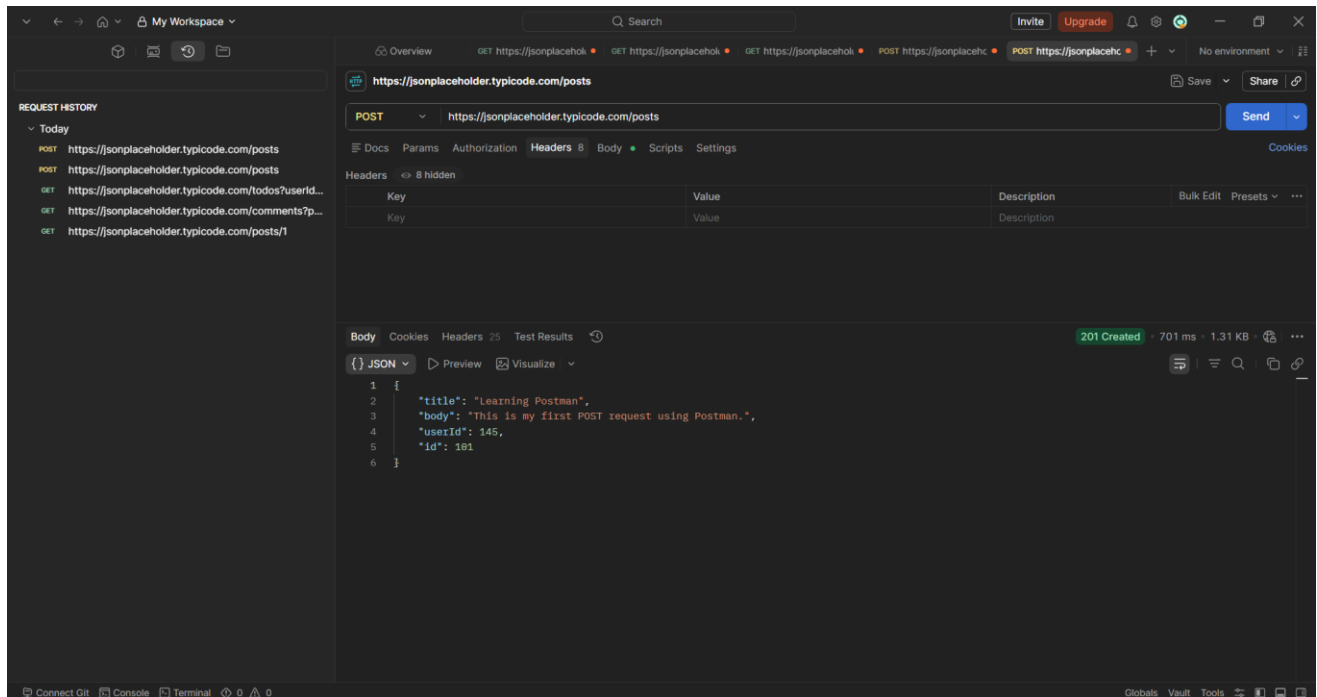
NEEL VAGHELA

d) Record the status code and the new id the server assigned.



TASK 4 Headers vs. No Headers

- a) Repeat Task 3's POST request, but this time remove the Content-Type header before sending.



- b) Compare the response to Task 3 — is it different? Record what you observe.

Feature	Task 3	Task 4
Method	POST	POST
Content-Type Header	Present	Removed
Request Body	JSON	JSON
Status Code	201 Created	Usually 201 Created
Response	Returns new object with <code>id</code>	Usually the same response
Real APIs	Works correctly	May fail without the header

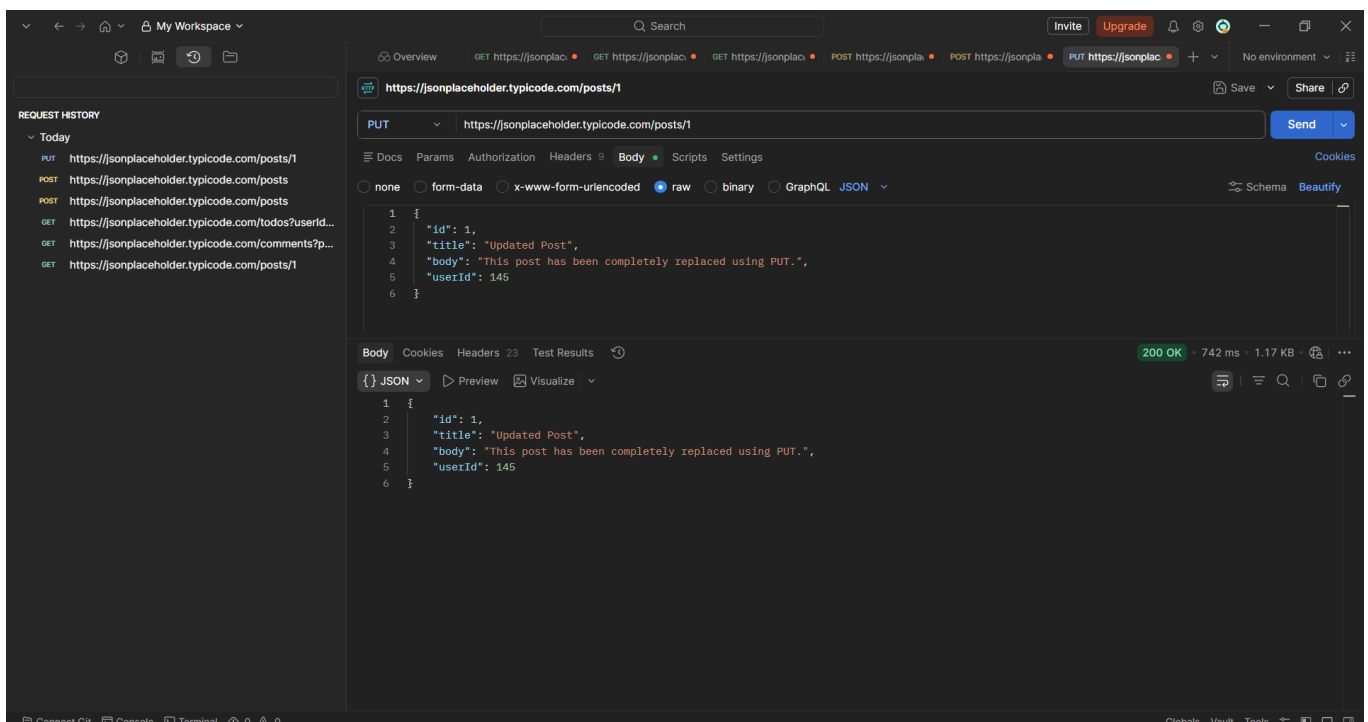
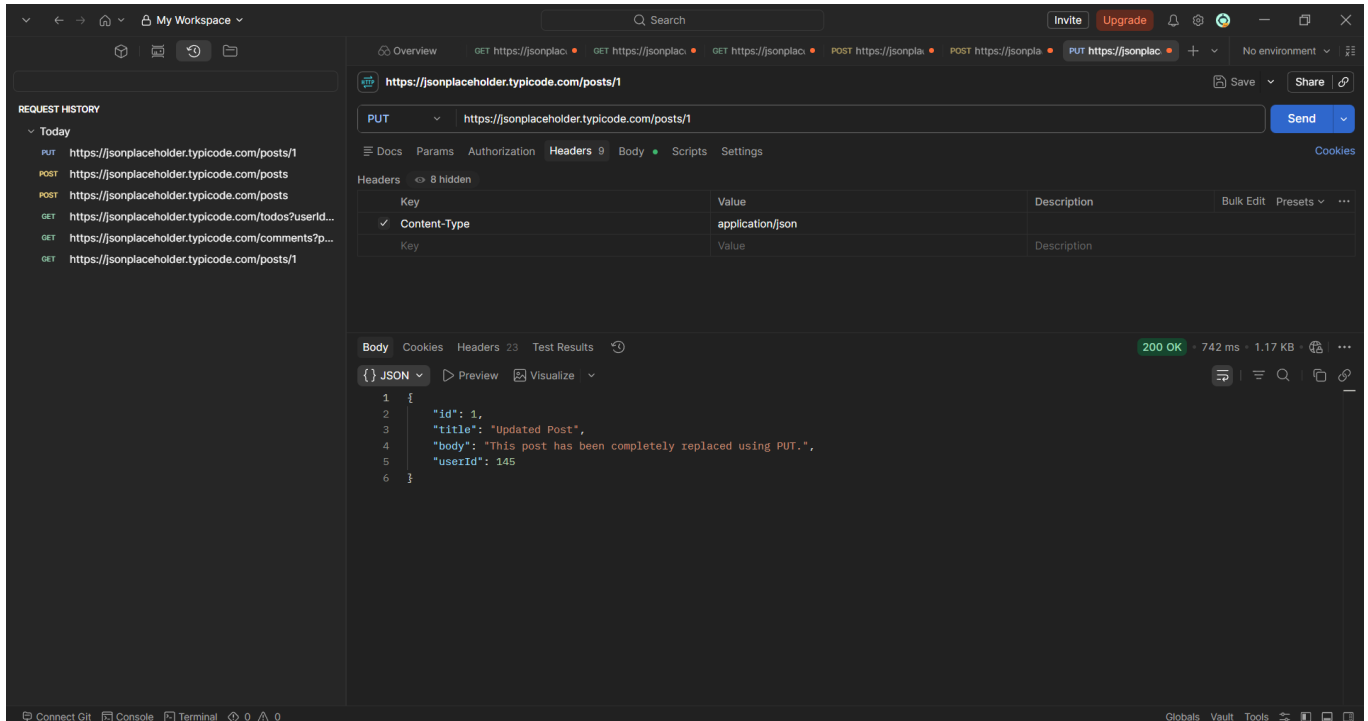
c) In 2–3 sentences, explain why headers matter for a request like this, even though this practice API is forgiving about it.

→ Headers tell the server important information about the request, such as the format of the data being sent. The **Content-Type: application/json** header lets the server know that the request body contains JSON. JSONPlaceholder accepts requests without this header because it is a practice API, but many real APIs would reject the request or fail to process the data correctly if the header were missing.

NEEL VAGHELA

TASK 5 PUT vs. PATCH

- a) Send a PUT request to /posts/1 that replaces the entire post (include id, title, body, and userId in the body).



- b) Send a PATCH request to the same /posts/1 that changes only the title.

The image displays two screenshots of a REST client interface, likely Postman, showing a PATCH request and its response.

Top Screenshot: The interface shows a PATCH request to `https://jsonplaceholder.typicode.com/posts/1`. The request body is a JSON object: `{ "title": "Updated Only Title" }`. The response is a 200 OK status with a response time of 1.21 s and a body size of 1.24 KB. The response body is a JSON object: `{ "userId": 1, "id": 1, "title": "Updated Only Title", "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto" }`.

Bottom Screenshot: The interface shows the same PATCH request, but the response body is a JSON object: `{ "userId": 1, "id": 1, "title": "Updated Only Title", "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto" }`. The response is a 200 OK status with a response time of 1.21 s and a body size of 1.24 KB.

- c) In one sentence, explain the difference between what PUT sent and what PATCH sent.
- PUT sends the complete resource to replace it, whereas PATCH sends only the fields that need to be updated.

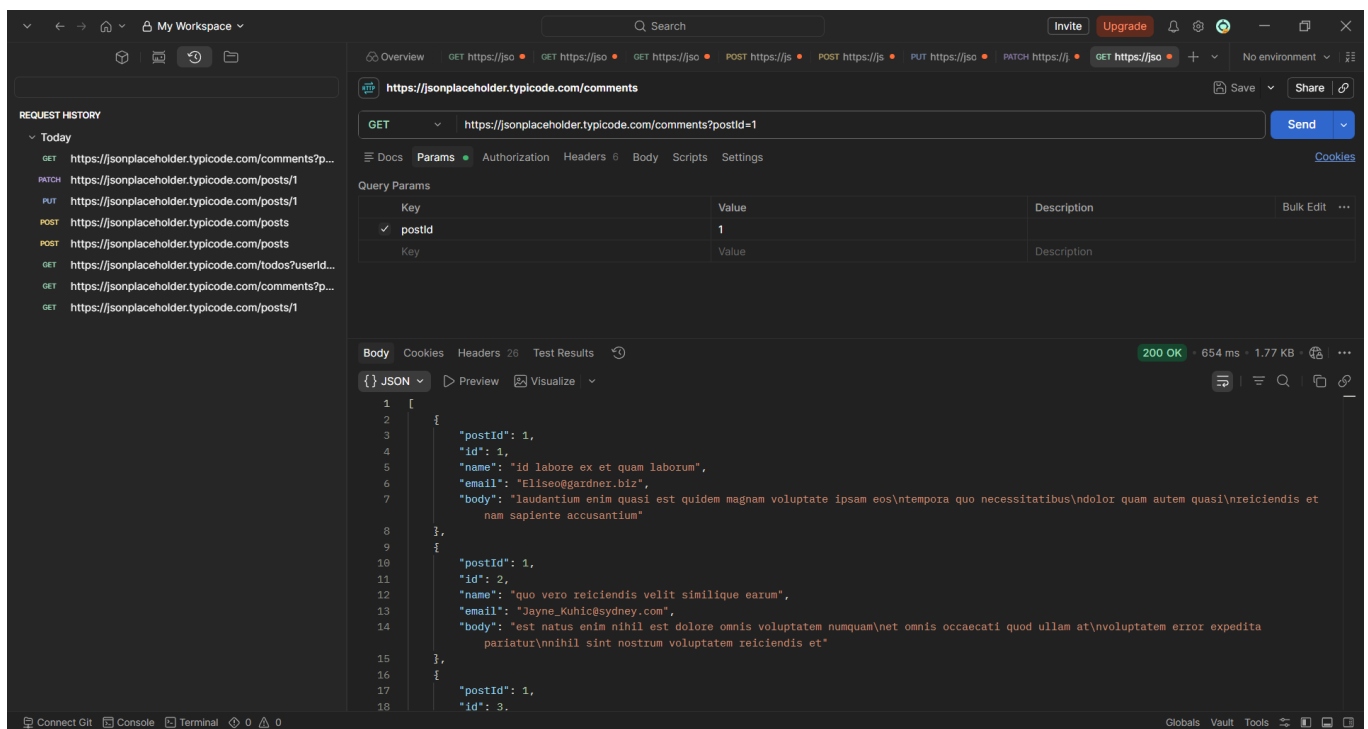
Final Challenge — A Realistic Request Flow

TASK 6 The Mini Feedback System

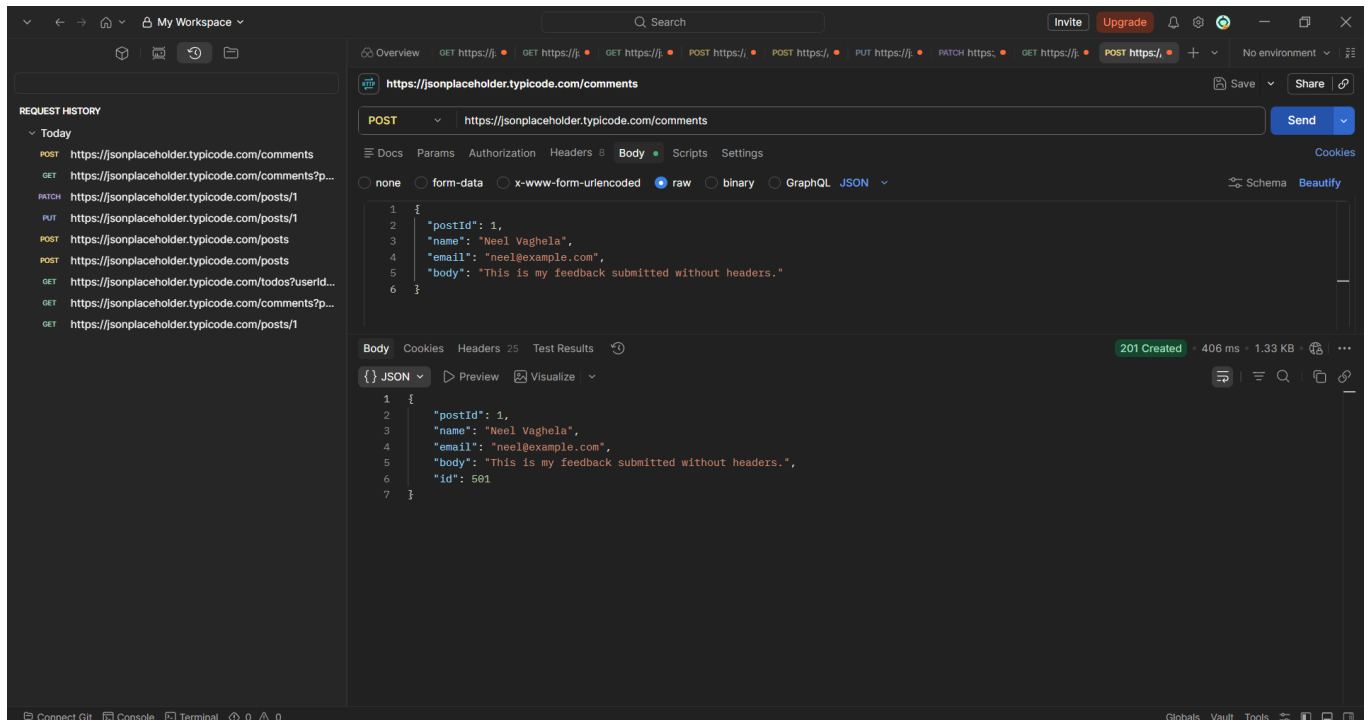
Scenario: you're testing the backend for a simple feedback form, where users read existing comments on a post and can leave their own.

Complete the following steps, in order:

- Send a GET request to /comments, using a query parameter to fetch only the comments for postId = 1.

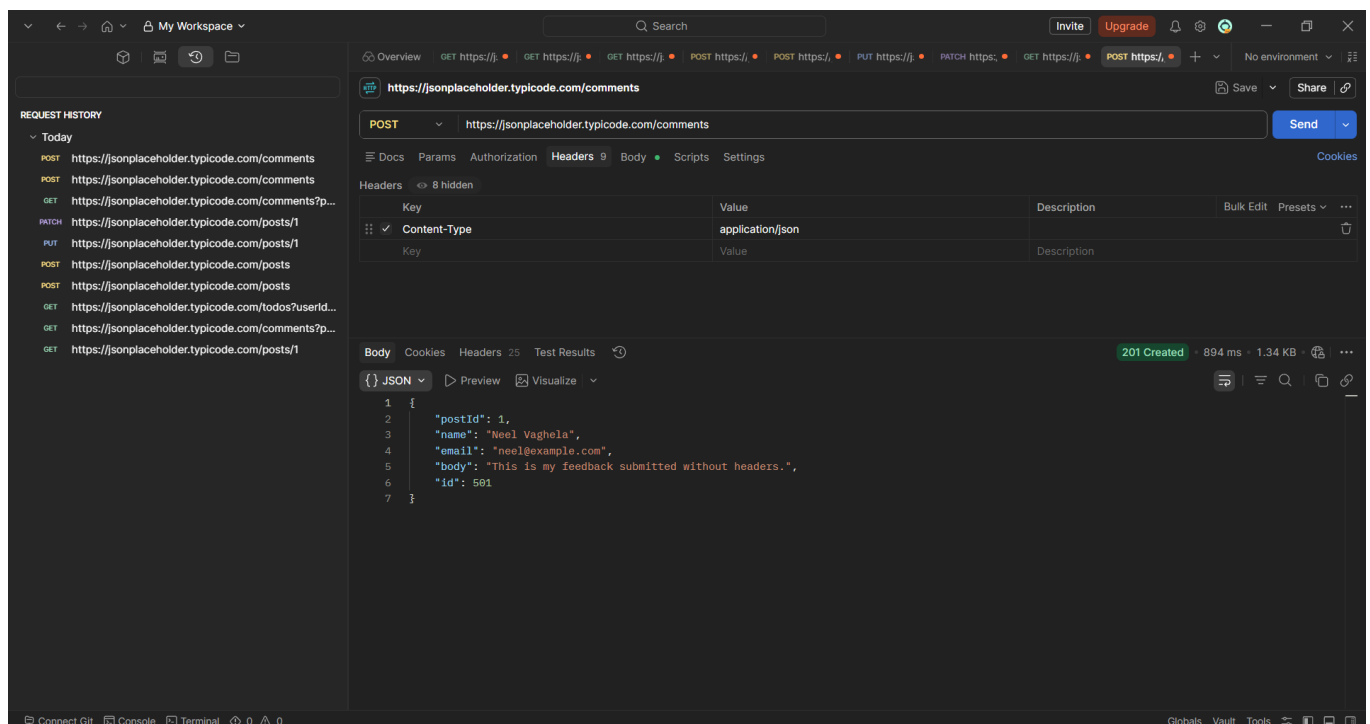


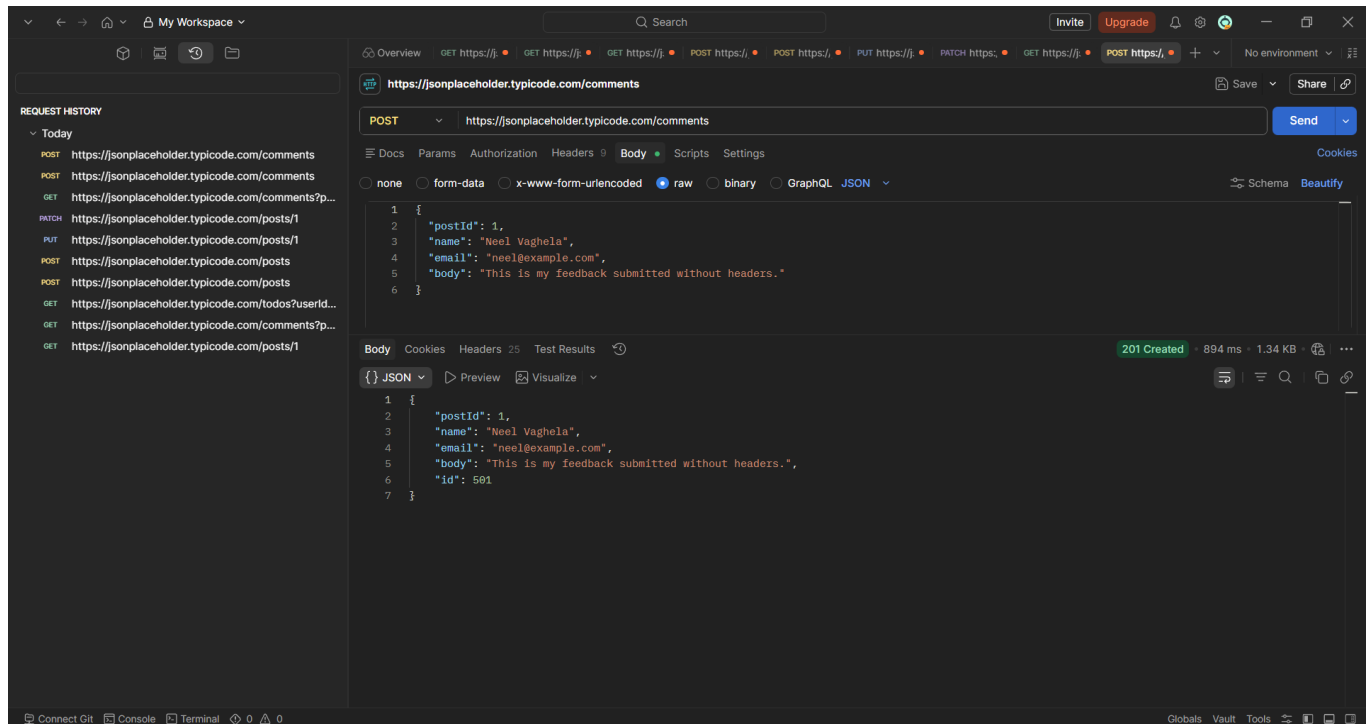
- Send a POST request to /comments to add a new comment — with no headers at all. Record the status code.



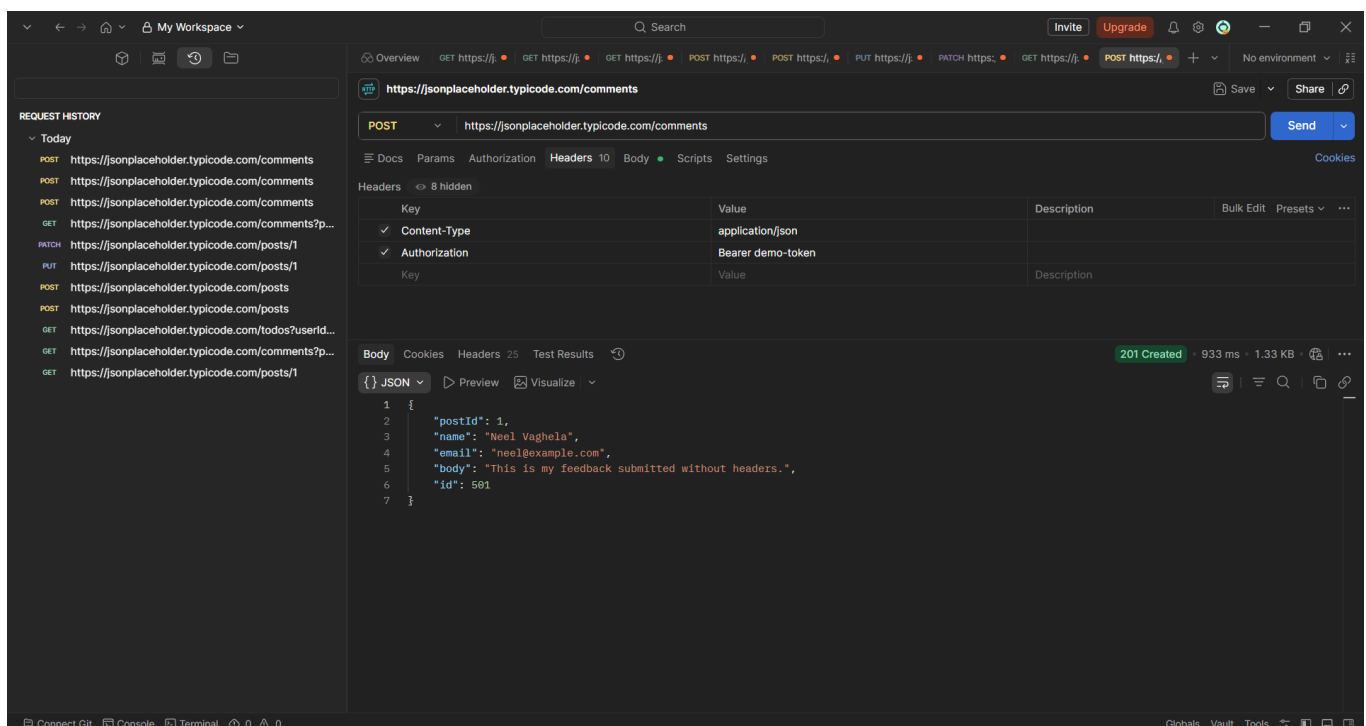
c) Resend the same POST from step (b), this time with the Content-Type header included. Compare the two responses.

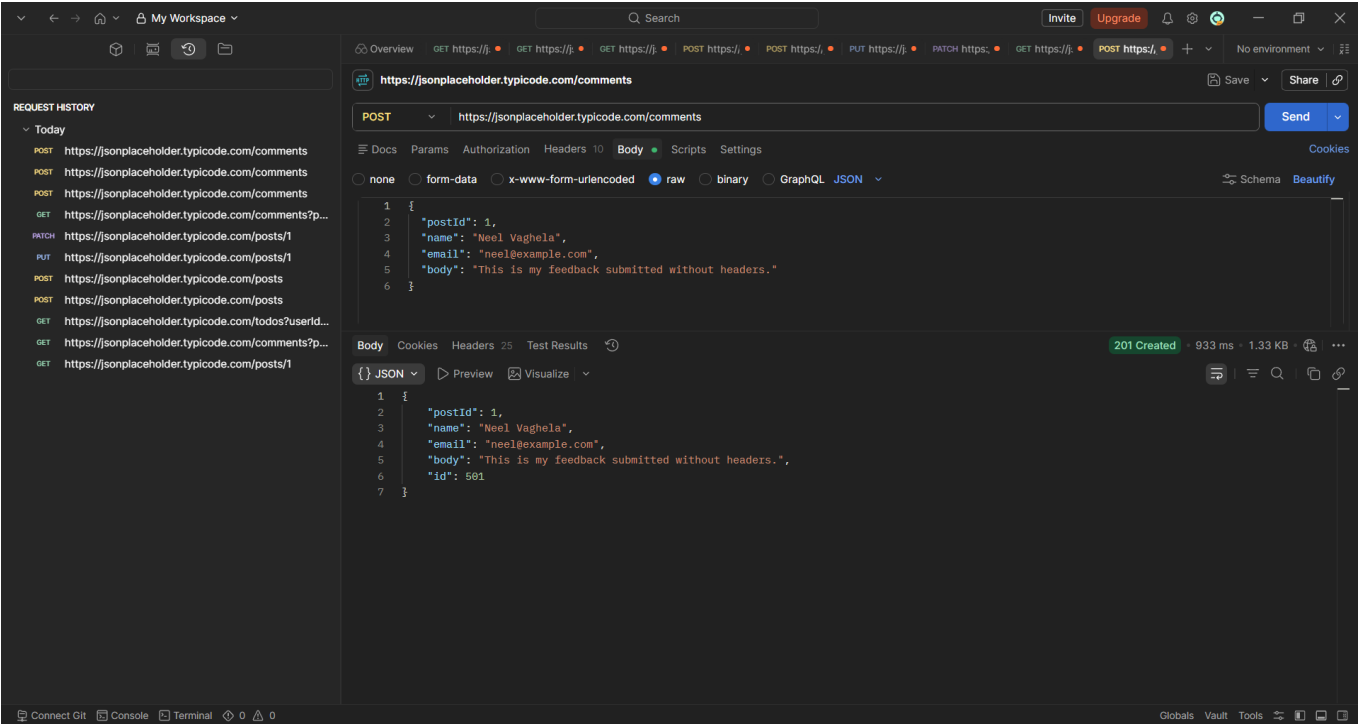
→ Both requests returned the same response and status code because JSONPlaceholder accepts the request even without the Content-Type header.





- d) Add one more header to this request: Authorization: Bearer demo-token — to simulate a login-protected submission, even though this practice API won't actually check it.





e) Record the status code for every step above (a–d) in a short table or list.

Step	Request	Status Code
a	GET /comments?postId=1	200 OK
b	POST /comments (without custom headers)	201 Created
c	POST /comments (with Content-Type)	201 Created
d	POST /comments (with Content-Type and Authorization)	201 Created

f) In 2–3 sentences, explain what would happen differently at step (d) if this were a real API that actually required authentication.

→ A real API would verify the Authorization token before processing the request. If the token was valid, the request would be accepted and the comment would be created. If the token was missing, expired, or invalid, the server would return an error such as 401 Unauthorized or 403 Forbidden, and the request would not be processed.